```sql
-- Retrieve the number of unique customers by country and city along with their average purchase value.

WITH a AS (
    SELECT c.country, c.city, t.customer_id,
        SUM(t.invoice_total) AS total_spent
    FROM customers c
    JOIN transactions t
        ON c.`Customer ID` = t.customer_id
    GROUP BY c.country, c.city, t.customer_id
)
SELECT
    country, city,
    COUNT(DISTINCT customer_id) AS unique_customers,
    AVG(total_spent) AS avg_purchase_value
FROM a
GROUP BY country, city;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

country	city	unique_customers	avg_purchase_value
Deutschland	Ahrensburg	1077	278.786435
Deutschland	Altona	1187	273.075872
Deutschland	Bad Homburg	663	273.707888
Deutschland	Barsinghausen	20	264.316000
Deutschland	Bergedorf	1146	276.417417

```sql
-- Calculate the percentage of returns compared to total transactions and track how it changes monthly.

select
    DATE_FORMAT(transaction_date, '%Y-%m-01') as month,
    count(*) as total_transactions,
    sum(case when invoice_total < 0 then 1 else 0 end) as return_transactions,
    sum(case when invoice_total < 0 then 1 else 0 end) * 100.0 / count(*) as return_percentage
from transactions
group by DATE_FORMAT(transaction_date, '%Y-%m-01')
order by month;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

month	total_transactions	return_transactions	return_percentage
2023-01-01	171759	9256	5.38895
2023-02-01	119380	6501	5.44564
2023-03-01	264005	14322	5.42490
2023-04-01	187136	9986	5.33623
2023-05-01	213135	11215	5.26192
2023-06-01	132241	6893	5.21245
2023-07-01	147591	7725	5.23406
2023-08-01	128408	6705	5.22164

```sql
-- Identify the top categories by total revenue and their share in overall sales.
SELECT
    p.category,
    SUM(t.invoice_total) AS total_revenue,
    ROUND(
        SUM(t.invoice_total) /
        (SELECT SUM(invoice_total) FROM transactions) * 100,
        2
    ) AS revenue_share_pct
FROM transactions t
JOIN products p
    ON t.product_id = p.product_id
GROUP BY p.category
ORDER BY total_revenue DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	total_revenue	revenue_share_pct
Feminine	717363055.04	45.91
Masculine	679201290.75	43.46
Children	166103991.08	10.63

```sql
-- Compare total sales volume and average basket value across payment methods.

with a as (
    select
        payment_method, invoice_id, sum(line_total) basket_value
    from transactions
    where transaction_type = 'Sale'
    group by payment_method, invoice_id
)

select payment_method, sum(basket_value) total_sales, avg(basket_value) avg_basket_value
from a
group by payment_method;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

payment_method	total_sales	avg_basket_value
Cash	154546512.24	154546512.240000
Credit Card	621514010.38	621514010.380000

```sql
-- Compare sales volume and revenue during discount periods vs non-discount periods.

select
    case
        when discount > 0 then 'Discount Period'
        else 'Non-Discount Period'
    end as period_type,
    sum(quantity) sales_volume,
    sum(line_total) total_revenue
from transactions
where transaction_type = 'Sale'
group by period_type;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

period_type	sales_volume	total_revenue
Non-Discount Period	4717992	631904692.20
Discount Period	1968132	144155830.42

```sql
-- Measure proportion of new vs repeat customers each month with average spend.

with first_purchase as (
    select
        customer_id, min(transaction_date) first_purchase_date
    from transactions
    where transaction_type = 'Sale'
    group by customer_id
),
monthly_sales as (
    select t.customer_id, date_format(t.transaction_date, '%Y-%m') month, sum(t.line_total) total_spend, f.first_purchase_date
    from transactions t
    join first_purchase f
        on t.customer_id = f.customer_id
    where t.transaction_type = 'Sale'
    group by t.customer_id, month, f.first_purchase_date
)
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

month	customer_type	customers	avg_spend
2023-01	New	109378	135.251510
2023-02	New	68586	155.031124
2023-02	Repeat	8444	167.176972
2023-03	New	134918	193.652760
2023-03	Repeat	27826	213.008645

```sql
-- Identify top and bottom stores by revenue and return rates.

with a as (
    select
        s.store_id,
        s.store_name,
        sum(case when t.transaction_type = 'Sale' then t.line_total else 0 end) revenue,
        sum(case when t.transaction_type = 'Return' then t.line_total else 0 end) returns
    from stores s
    join transactions t
        on s.store_id = t.store_id
    group by s.store_id, s.store_name
)

select *,
       dense_rank() over(order by revenue desc) revenue_rank,
       dense_rank() over(order by returns desc) return_rank
from a;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

store_id	store_name	revenue	returns	revenue_rank	return_rank
18	Store Glasgow	2405985.27	-135743.25	35	1
19	Store Liverpool	2647766.36	-150627.27	34	2
33	Store Braga	2843220.19	-156985.31	33	3
28	Store Valencia	2982464.34	-171324.63	32	4
15	Store Frankfurt am Main	3235612.37	-176204.03	31	5

```sql
-- Segment products into price bands and analyze revenue and returns.

with a as (
    select
        case
            when unit_price < 50 then 'Low'
            when unit_price between 50 and 150 then 'Medium'
            else 'High'
        end as price_band,
        transaction_type, line_total
    from transactions
)
select
    price_band,
    sum(case when transaction_type = 'Sale' then line_total else 0 end) revenue,
    sum(case when transaction_type = 'Return' then line_total else 0 end) returns
from a
group by price_band;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

price_band	revenue	returns
Medium	125452513.89	-7025771.97
Low	89166514.39	-5014298.16
High	561441494.34	-31275577.50

```sql
-- Calculate average number of items and revenue per basket across countries.

with a as (
    select
        s.country, t.invoice_id, sum(t.quantity) total_items, sum(t.line_total) basket_revenue
    from transactions t
    join stores s
        on t.store_id = s.store_id
    where t.transaction_type = 'Sale'
    group by s.country, t.invoice_id
)

select
    country, avg(total_items) avg_items_per_basket, avg(basket_revenue) avg_revenue_per_basket
from a
group by country;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

country	avg_items_per_basket	avg_revenue_per_basket
United States	1758358.0000	79464205.730000
中国	1618960.0000	570307426.150000
Deutschland	781572.0000	31615480.260000
United Kingdom	653022.0000	18434022.000000
France	675725.0000	27378649.270000

```sql
-- Analyze month-over-month revenue growth overall and for the top three categories

with monthly_revenue as (
    select
        date_format(t.transaction_date, '%Y-%m') as month, d.category, sum(t.line_total) as revenue
    from transactions t
    left join discounts d
        on t.transaction_date between d.start_date and d.end_date
    where t.transaction_type = 'Sale'
    group by month, d.category
),
category_totals as (
    select
        category,
        sum(revenue) as total_revenue
    from monthly_revenue
    group by category
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

month	category	revenue	mom_growth
2023-01	Feminine	12261661.40	NULL
2023-03	Feminine	38170218.16	25908556.76
2023-05	Feminine	43525199.25	5354981.09
2023-09	Feminine	58947714.00	15422514.75
2023-10	Feminine	34681866.80	-24265847.20